import React, { useState } from 'react'; import { ChevronDown, ChevronRight, Code, BookOpen, PlayCircle } from 'lucide-react';

const AIMLFundamentals = () => { const [expandedSection, setExpandedSection] = useState(null); const [selectedTopic, setSelectedTopic] = useState(null);

const topics = [{ id: 'python-basics', category: 'Python Fundamentals', title: '1. Python Core Concepts', importance: 'Critical Foundation', questions: ['What are Python data types and structures?', 'How do list comprehensions work?', 'What are lambda functions?', 'Explain decorators and their use'], explanation: `Python is the primary language for AI/ML. Understanding its core concepts is essential.

Key Concepts: • Data Types: int, float, str, bool, list, tuple, dict, set • Control Flow: if/else, loops, comprehensions • Functions: def, lambda, *args, **kwargs • OOP: classes, inheritance, encapsulation, code: # Python Basics for AI/ML

1. Data Structures

numbers = [1, 2, 3, 4, 5] data_dict = {'name': 'John', 'age': 25, 'scores': [90, 85, 88]}

2. List Comprehensions (Very important for data processing)

squares = [x**2 for x in range(10)] print("Squares:", squares)

Filter even numbers

evens = [x for x in numbers if x % 2 == 0] print("Even numbers:", evens)

3. Lambda Functions (Used in pandas, map/filter operations)

multiply = lambda x, y: x * y print("Lambda result:", multiply(5, 3))

4. Map, Filter, Reduce

from functools import reduce squared_nums = list(map(lambda x: x**2, numbers)) print("Mapped:", squared_nums)

5. Dictionary Operations (Essential for data manipulation)

student = {'name': 'Alice', 'grades': [85, 90, 92]} print("Average grade:", sum(student['grades']) / len(student['grades']))

6. Exception Handling (Important for robust ML pipelines)

try: result = 10 / 0 except ZeroDivisionError as e: print(f"Error handled: {e}") result = None

print("\n--- Output ---") print(f"List comprehension creates efficient loops: {squares}") print(f"Lambda functions are anonymous functions: {multiply(4, 5)}") `}, { id: 'numpy', category: 'NumPy', title: '2. NumPy - Numerical Computing', importance: 'Essential for ML', questions: [ 'What is array broadcasting?', 'How to perform vectorized operations?', 'What are array indexing and slicing?', 'Explain reshaping and array manipulation' ], explanation:` NumPy is the foundation for numerical computing in Python. All ML libraries use NumPy arrays.

Key Concepts: • Arrays: Faster than Python lists • Broadcasting: Operations on different shaped arrays • Vectorization: Avoid loops, use array operations • Linear Algebra: Matrix operations for ML`, code:` import numpy as np

1. Creating Arrays

arr1d = np.array([1, 2, 3, 4, 5]) arr2d = np.array([[1, 2, 3], [4, 5, 6]]) zeros = np.zeros((3, 3)) ones = np.ones((2, 4)) random_arr = np.random.rand(3, 3)

print("1D Array:", arr1d) print("2D Array:\n", arr2d) print("Random Array:\n", random_arr)

2. Array Operations (Vectorized - FAST!)

a = np.array([1, 2, 3, 4]) b = np.array([10, 20, 30, 40])

print("\nElement-wise addition:", a + b) print("Element-wise multiplication:", a * b) print("Scalar operations:", a * 2)

3. Broadcasting (Very Important!)

matrix = np.array([[1, 2, 3], [4, 5, 6]]) vector = np.array([10, 20, 30]) result = matrix + vector # Vector broadcasts to each row print("\nBroadcasting result:\n", result)

4. Indexing and Slicing

arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]]) print("\nSlicing [0:2, 1:3]:\n", arr[0:2, 1:3]) print("Boolean indexing:", arr[arr > 5])

5. Statistical Operations (Used in data analysis)

data = np.random.randn(100) # Normal distribution print("\nMean:", np.mean(data)) print("Std Dev:", np.std(data)) print("Min:", np.min(data)) print("Max:", np.max(data))

6. Reshaping (Critical for ML models)

original = np.array([1, 2, 3, 4, 5, 6]) reshaped = original.reshape(2, 3) print("\nReshaped (2,3):\n", reshaped)

7. Matrix Operations (Linear Algebra for ML)

A = np.array([[1, 2], [3, 4]]) B = np.array([[5, 6], [7, 8]]) print("\nMatrix multiplication:\n", np.dot(A, B)) print("Transpose:\n", A.T)

8. Aggregations

matrix = np.random.randint(1, 10, (3, 4)) print("\nMatrix:\n", matrix) print("Sum of all elements:", np.sum(matrix)) print("Sum along axis 0 (columns):", np.sum(matrix, axis=0)) print("Sum along axis 1 (rows):", np.sum(matrix, axis=1)) }, { id: 'pandas', category: 'Pandas', title: '3. Pandas - Data Manipulation', importance: 'Critical for Data Processing', questions: ['How to handle missing data?', 'What is groupby and aggregation?', 'How to merge/join DataFrames?', 'What are apply, map, and applymap?'], explanation:Pandas is essential for data preprocessing, the most time-consuming part of ML projects.

Key Concepts: • DataFrame: 2D labeled data structure • Series: 1D labeled array • Data Cleaning: Handle missing values, duplicates • Grouping & Aggregation: Summarize data • Merging: Combine datasets, code: import pandas as pd import numpy as np

1. Creating DataFrames

data = { 'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'], 'Age': [25, 30, 35, np.nan, 28], 'Salary': [50000, 60000, 75000, 80000, 55000], 'Department': ['IT', 'HR', 'IT', 'Finance', 'HR'] } df = pd.DataFrame(data) print("DataFrame:\n", df)

2. Basic Operations

print("\nFirst 3 rows:\n", df.head(3)) print("\nDataFrame Info:") print(df.info()) print("\nStatistical Summary:\n", df.describe())

3. Handling Missing Data (Very Important!)

print("\nMissing values:\n", df.isnull().sum()) df['Age'].fillna(df['Age'].mean(), inplace=True) print("After filling missing values:\n", df)

4. Filtering Data

it_employees = df[df['Department'] == 'IT'] print("\nIT Department:\n", it_employees)

high_earners = df[df['Salary'] > 60000] print("\nHigh earners:\n", high_earners)

5. GroupBy (Essential for aggregations)

dept_stats = df.groupby('Department').agg({ 'Salary': ['mean', 'min', 'max'], 'Age': 'mean' }) print("\nDepartment Statistics:\n", dept_stats)

6. Apply Function (Transform data)

df['Salary_Category'] = df['Salary'].apply(lambda x: 'High' if x > 65000 else 'Medium' if x > 55000 else 'Low') print("\nWith Salary Category:\n", df)

7. Creating new columns

df['Age_Group'] = pd.cut(df['Age'], bins=[0, 30, 40, 100], labels=['Young', 'Middle', 'Senior']) print("\nWith Age Groups:\n", df)

8. Merging DataFrames

df2 = pd.DataFrame({ 'Department': ['IT', 'HR', 'Finance'], 'Budget': [100000, 50000, 150000] }) merged = pd.merge(df, df2, on='Department', how='left') print("\nMerged DataFrame:\n", merged)

9. Sorting and Ranking

sorted_df = df.sort_values('Salary', ascending=False) print("\nSorted by Salary:\n", sorted_df)

10. String Operations

df['Name_Upper'] = df['Name'].str.upper() df['Name_Length'] = df['Name'].str.len() print("\nString Operations:\n", df[['Name', 'Name_Upper', 'Name_Length']]) `}, { id: 'matplotlib', category: 'Matplotlib', title: '4. Matplotlib - Data Visualization', importance: 'Essential for EDA', questions: [ 'How to create multiple subplots?', 'What are figure and axes objects?', 'How to customize plots?', 'What are different plot types?' ], explanation:` Matplotlib is the foundation for data visualization in Python. Understanding plots is crucial for Exploratory Data Analysis (EDA).

Key Concepts: • Figure & Axes: Plot containers • Plot Types: Line, scatter, bar, histogram, box • Customization: Colors, labels, legends, styles • Subplots: Multiple plots in one figure`, code:` import matplotlib.pyplot as plt

import numpy as np

1. Basic Line Plot

x = np.linspace(0, 10, 100) y1 = np.sin(x) y2 = np.cos(x)

plt.figure(figsize=(10, 6)) plt.plot(x, y1, label='sin(x)', color='blue', linewidth=2) plt.plot(x, y2, label='cos(x)', color='red', linewidth=2, linestyle='--') plt.title('Trigonometric Functions', fontsize=16) plt.xlabel('X axis', fontsize=12) plt.ylabel('Y axis', fontsize=12) plt.legend() plt.grid(True, alpha=0.3) plt.show()

2. Scatter Plot (Used for correlation analysis)

np.random.seed(42) x = np.random.randn(100) y = 2 * x + np.random.randn(100) * 0.5

plt.figure(figsize=(8, 6)) plt.scatter(x, y, c=y, cmap='viridis', alpha=0.6, edgecolors='black') plt.colorbar(label='Y values') plt.title('Scatter Plot with Color Mapping') plt.xlabel('X values') plt.ylabel('Y values') plt.show()

3. Histogram (Distribution analysis)

data = np.random.randn(1000) plt.figure(figsize=(10, 6)) plt.hist(data, bins=30, alpha=0.7, color='skyblue', edgecolor='black') plt.axvline(data.mean(), color='red', linestyle='--', label=f'Mean: {data.mean():.2f}') plt.title('Distribution of Random Data') plt.xlabel('Value') plt.ylabel('Frequency') plt.legend() plt.show()

4. Subplots (Multiple visualizations)

fig, axes = plt.subplots(2, 2, figsize=(12, 10))

Plot 1: Line plot

axes[0, 0].plot(x, y1, 'b-') axes[0, 0].set_title('Line Plot') axes[0, 0].grid(True)

Plot 2: Scatter

axes[0, 1].scatter(x, y, alpha=0.5) axes[0, 1].set_title('Scatter Plot')

Plot 3: Bar plot

categories = ['A', 'B', 'C', 'D'] values = [23, 45, 56, 78] axes[1, 0].bar(categories, values, color=['red', 'blue', 'green', 'orange']) axes[1, 0].set_title('Bar Plot')

Plot 4: Box plot

data_box = [np.random.normal(0, std, 100) for std in range(1, 4)] axes[1, 1].boxplot(data_box, labels=['Group 1', 'Group 2', 'Group 3']) axes[1, 1].set_title('Box Plot')

plt.tight_layout() plt.show()

5. Advanced Customization

fig, ax = plt.subplots(figsize=(10, 6)) x = np.linspace(0, 10, 50) y = np.sin(x)

ax.plot(x, y, marker='o', linestyle='-', color='purple', markersize=6, markerfacecolor='yellow', markeredgecolor='black') ax.fill_between(x, y, alpha=0.3, color='purple') ax.set_title('Advanced Plot Styling', fontsize=18, fontweight='bold') ax.set_xlabel('X axis', fontsize=14) ax.set_ylabel('Y axis', fontsize=14) ax.spines['top'].set_visible(False) ax.spines['right'].set_visible(False) ax.grid(True, linestyle=':', alpha=0.5) plt.show()

print("Matplotlib plots displayed!") }, { id: 'seaborn', category: 'Seaborn', title: '5. Seaborn - Statistical Visualization', importance: 'Advanced EDA', questions: ['What is the difference between Seaborn and Matplotlib?', 'How to create correlation heatmaps?', 'What are pairplots and when to use them?', 'How to visualize distributions?'], explanation:Seaborn provides high-level interface for statistical graphics, built on Matplotlib.

Key Concepts: • Statistical Plots: Distribution, regression, categorical • Themes: Beautiful default styles • Complex Visualizations: Heatmaps, pairplots, facetgrids • Integration: Works seamlessly with pandas, code: import seaborn as sns import matplotlib.pyplot as plt import numpy as np import pandas as pd

Set style

sns.set_style("whitegrid")

1. Distribution Plots

np.random.seed(42) data = np.random.randn(1000)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

Histogram with KDE

sns.histplot(data, kde=True, ax=axes[0], color='skyblue') axes[0].set_title('Histogram with KDE')

Violin plot

df = pd.DataFrame({ 'Category': np.random.choice(['A', 'B', 'C'], 300), 'Value': np.random.randn(300) }) sns.violinplot(data=df, x='Category', y='Value', ax=axes[1]) axes[1].set_title('Violin Plot') plt.tight_layout() plt.show()

2. Correlation Heatmap (Very Important for Feature Analysis!)

df_corr = pd.DataFrame(np.random.randn(100, 5), columns=['Feature1', 'Feature2', 'Feature3', 'Feature4', 'Target']) df_corr['Target'] = (0.7 * df_corr['Feature1'] + 0.3 * df_corr['Feature2'] + np.random.randn(100) * 0.1)

plt.figure(figsize=(10, 8)) correlation_matrix = df_corr.corr() sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', center=0, square=True, linewidths=1) plt.title('Correlation Heatmap', fontsize=16) plt.show()

3. Pairplot (Relationships between features)

iris_like = pd.DataFrame({ 'sepal_length': np.random.uniform(4, 8, 150), 'sepal_width': np.random.uniform(2, 5, 150), 'petal_length': np.random.uniform(1, 7, 150), 'species': np.random.choice(['A', 'B', 'C'], 150) }) iris_like['petal_width'] = iris_like['petal_length'] * 0.3 + np.random.randn(150) * 0.1

sns.pairplot(iris_like, hue='species', diag_kind='kde', height=2.5) plt.suptitle('Pairplot for Feature Relationships', y=1.02) plt.show()

4. Box Plots and Swarm Plots

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

sns.boxplot(data=df, x='Category', y='Value', ax=axes[0]) axes[0].set_title('Box Plot by Category')

sns.swarmplot(data=df, x='Category', y='Value', ax=axes[1], alpha=0.6) axes[1].set_title('Swarm Plot') plt.tight_layout() plt.show()

5. Regression Plot (Shows relationship with confidence interval)

x = np.random.randn(100) y = 2 * x + np.random.randn(100) * 0.5

plt.figure(figsize=(10, 6)) sns.regplot(x=x, y=y, scatter_kws={'alpha':0.5}, line_kws={'color':'red'}) plt.title('Regression Plot with Confidence Interval') plt.xlabel('X') plt.ylabel('Y') plt.show()

6. Countplot (For categorical data)

categories = pd.DataFrame({ 'Category': np.random.choice(['Type A', 'Type B', 'Type C', 'Type D'], 200) })

plt.figure(figsize=(10, 6)) sns.countplot(data=categories, x='Category', palette='Set2') plt.title('Count of Each Category') plt.ylabel('Frequency') plt.show()

print("Seaborn plots displayed!") }, { id: 'statistics', category: 'Statistics', title: '6. Statistics for ML', importance: 'Mathematical Foundation', questions: ['What is the

difference between mean, median, and mode?', 'What are variance and standard deviation?', 'What is correlation vs causation?', 'What are probability distributions?', 'What is the Central Limit Theorem?'], explanation:Statistics is the mathematical foundation of Machine Learning. Understanding statistical concepts is crucial for model evaluation and interpretation.

Key Concepts: • Descriptive Statistics: Mean, median, mode, variance, std • Probability: Distributions, independence, conditional • Hypothesis Testing: p-values, significance • Correlation: Relationships between variables • Sampling: Representative data collection, code: import numpy as np import pandas as pd from scipy import stats import matplotlib.pyplot as plt

Generate sample data

np.random.seed(42) data = np.random.normal(100, 15, 1000) # Mean=100, Std=15

1. Measures of Central Tendency

mean = np.mean(data) median = np.median(data) mode_result = stats.mode(data.round(), keepdims=True) mode = mode_result.mode[0]

print("=== CENTRAL TENDENCY ===") print(f"Mean: {mean:.2f}") print(f"Median: {median:.2f}") print(f"Mode: {mode:.2f}")

2. Measures of Spread

variance = np.var(data) std_dev = np.std(data) range_val = np.max(data) - np.min(data) iqr = np.percentile(data, 75) - np.percentile(data, 25)

print("\n=== MEASURES OF SPREAD ===") print(f"Variance: {variance:.2f}") print(f"Standard Deviation: {std_dev:.2f}") print(f"Range: {range_val:.2f}") print(f"IQR (Interquartile Range): {iqr:.2f}")

3. Percentiles

percentiles = [25, 50, 75, 90, 95] print("\n=== PERCENTILES ===") for p in percentiles: print(f"{p}th percentile: {np.percentile(data, p):.2f}")

4. Correlation

x = np.random.randn(100) y = 2 * x + np.random.randn(100) * 0.5 # Strong positive correlation z = np.random.randn(100) # No correlation with x

correlation_xy = np.corrcoef(x, y)[0, 1] correlation_xz = np.corrcoef(x, z)[0, 1]

print("\n=== CORRELATION ===") print(f"Correlation between x and y: {correlation_xy:.3f}") print(f"Correlation between x and z: {correlation_xz:.3f}")

5. Probability Distributions

print("\n=== PROBABILITY DISTRIBUTIONS ===")

Normal Distribution

normal_data = np.random.normal(0, 1, 1000) print(f"Normal distribution - Mean: {np.mean(normal_data):.3f}, Std: {np.std(normal_data):.3f}")

Binomial Distribution (coin flips)

binomial_data = np.random.binomial(10, 0.5, 1000) # 10 flips, 0.5 probability print(f"Binomial (10 flips) - Mean successes: {np.mean(binomial_data):.3f}")

6. Hypothesis Testing (t-test)

group1 = np.random.normal(100, 10, 100) group2 = np.random.normal(105, 10, 100)

t_stat, p_value = stats.ttest_ind(group1, group2)

print("\n=== HYPOTHESIS TESTING (t-test) ===") print(f"t-statistic: {t_stat:.4f}") print(f"p-value: {p_value:.4f}") print(f"Significant difference (p < 0.05): {p_value < 0.05}")

7. Z-Score (Standardization)

z_scores = (data - mean) / std_dev print("\n=== Z-SCORES ===") print(f"Z-score of value 115: {((115 - mean) / std_dev):.3f}") print(f"Values beyond 2 std devs: {np.sum(np.abs(z_scores) > 2)}")

8. Confidence Interval

confidence = 0.95 n = len(data) std_error = std_dev / np.sqrt(n) margin_error = std_error * stats.t.ppf((1 + confidence) / 2, n - 1)

print("\n=== CONFIDENCE INTERVAL (95%) ===") print(f"Mean: {mean:.2f}") print(f"CI: ({mean - margin_error:.2f}, {mean + margin_error:.2f})")

Visualization

fig, axes = plt.subplots(2, 2, figsize=(14, 10))

Plot 1: Histogram with statistics

axes[0, 0].hist(data, bins=30, alpha=0.7, color='skyblue', edgecolor='black') axes[0, 0].axvline(mean, color='red', linestyle='--', label=f'Mean: {mean:.2f}') axes[0, 0].axvline(median, color='green', linestyle='--', label=f'Median: {median:.2f}') axes[0, 0].legend() axes[0, 0].set_title('Distribution with Central Tendency')

Plot 2: Box plot

axes[0, 1].boxplot(data, vert=True) axes[0, 1].set_title('Box Plot (shows quartiles and outliers)') axes[0, 1].set_ylabel('Values')

Plot 3: Correlation scatter

axes[1, 0].scatter(x, y, alpha=0.5) axes[1, 0].set_title(f'Strong Correlation (r={correlation_xy:.2f})') axes[1, 0].set_xlabel('X') axes[1, 0].set_ylabel('Y')

Plot 4: Normal distribution

axes[1, 1].hist(normal_data, bins=30, alpha=0.7, color='lightgreen', edgecolor='black', density=True) xmin, xmax = axes[1, 1].get_xlim() x_norm = np.linspace(xmin, xmax, 100) p = stats.norm.pdf(x_norm, 0, 1) axes[1, 1].plot(x_norm, p, 'r-', linewidth=2, label='Normal PDF') axes[1, 1].set_title('Normal Distribution') axes[1, 1].legend()

plt.tight_layout() plt.show()

print("\n✓ Statistical analysis complete!") `}, { id: 'linear-algebra', category: 'Mathematics', title: '7. Linear Algebra for ML', importance: 'Core Mathematical Foundation', questions: ['What are vectors and matrices?', 'What is matrix multiplication?', 'What are eigenvalues and eigenvectors?', 'What is the dot product?', 'How does dimensionality reduction work?'], explanation:` Linear Algebra is fundamental to ML. Neural networks, PCA, SVD, and many algorithms rely heavily on matrix operations.

Key Concepts: • Vectors: 1D arrays representing data points • Matrices: 2D arrays for transformations • Dot Product: Similarity between vectors • Matrix Multiplication: Combine transformations • Eigenvalues/Eigenvectors: Principal components • Inverse & Transpose: Matrix operations`, code:` import numpy as np import matplotlib.pyplot as plt

print("=== LINEAR ALGEBRA FOR ML ===\n")

1. Vectors and Basic Operations

v1 = np.array([1, 2, 3]) v2 = np.array([4, 5, 6])

print("1. VECTOR OPERATIONS") print(f"Vector v1: {v1}") print(f"Vector v2: {v2}") print(f"Addition: {v1 + v2}") print(f"Scalar multiplication: {3 * v1}") print(f"Element-wise multiplication: {v1 * v2}")

2. Dot Product (Measures similarity)

dot_product = np.dot(v1, v2) magnitude_v1 = np.linalg.norm(v1) magnitude_v2 = np.linalg.norm(v2) cosine_similarity = dot_product / (magnitude_v1 * magnitude_v2)

print(f"\n2. DOT PRODUCT & SIMILARITY") print(f"Dot product: {dot_product}") print(f"Magnitude of v1: {magnitude_v1:.3f}") print(f"Magnitude of v2: {magnitude_v2:.3f}") print(f"Cosine similarity: {cosine_similarity:.3f}")

3. Matrices

A = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

B = np.array([[9, 8, 7], [6, 5, 4], [3, 2, 1]])

print(f"\n3. MATRIX OPERATIONS") print(f"Matrix A:\n{A}") print(f"Transpose of A:\n{A.T}") print(f"Matrix addition A + B:\n{A + B}")

4. Matrix Multiplication (Key for Neural Networks!)

C = np.array([[1, 2], [3, 4], [5, 6]])

D = np.array([[7, 8, 9], [10, 11, 12]])

result = np.dot(C, D) print(f"\n4. MATRIX MULTIPLICATION") print(f"C (3x2):\n{C}") print(f"D (2x3):\n{D}") print(f"C × D (3x3):\n{result}")

5. Identity Matrix and Inverse

I = np.eye(3) A_invertible = np.array([[1, 2, 1], [2, 1, 2], [1, 1, 1]]) A_inv = np.linalg.inv(A_invertible)

print(f"\n5. IDENTITY & INVERSE") print(f"Identity matrix (3x3):\n{I}") print(f"Matrix A:\n{A_invertible}") print(f"Inverse of A:\n{A_inv}") print(f"A × A_inv (should be I):\n{np.dot(A_invertible, A_inv).round(10)}")

6. Eigenvalues and Eigenvectors (PCA uses this!)

matrix = np.array([[4, 2], [1, 3]])

eigenvalues, eigenvectors = np.linalg.eig(matrix)

print(f"\n6. EIGENVALUES & EIGENVECTORS") print(f"Matrix:\n{matrix}") print(f"Eigenvalues: {eigenvalues}") print(f"Eigenvectors:\n{eigenvectors}")

Verify: A × v = λ × v

for i in range(len(eigenvalues)): v = eigenvectors[:, i] lambda_val = eigenvalues[i] Av = np.dot(matrix, v) lambda_v = lambda_val * v print(f"\nVerification for eigenvalue {lambda_val:.3f}😊 print(f"A×v = {Av}") print(f"λ×v = {lambda_v}")

7. Determinant (tells if matrix is invertible)

det_A = np.linalg.det(matrix) print(f"\n7. DETERMINANT") print(f"Determinant of matrix: {det_A:.3f}") print(f"Matrix is invertible: {det_A != 0}")

8. Matrix Rank (dimensionality)

rank = np.linalg.matrix_rank(A) print(f"\n8. MATRIX RANK") print(f"Rank of A: {rank}") print(f"Matrix A is full rank: {rank == min(A.shape)}")

Visualization

fig, axes = plt.subplots(1, 2, figsize=(14, 6))

Plot 1: Vector visualization

axes[0].quiver(0, 0, v1[0], v1[1], angles='xy', scale_units='xy', scale=1, color='blue', width=0.01, label='v1') axes[0].quiver(0, 0, v2[0], v2[1], angles='xy', scale_units='xy', scale=1, color='red', width=0.01, label='v2') axes[0].quiver(0, 0, v1[0]+v2[0], v1[1]+v2[1], angles='xy', scale_units='xy', scale=1, color='green', width=0.01, label='v1+v2') axes[0].set_xlim(-1, 7) axes[0].set_ylim(-1, 9) axes[0].set_aspect('equal') axes[0].grid(True, alpha=0.3) axes[0].legend() axes[0].set_title('Vector Addition') axes[0].set_xlabel('X') axes[0].set_ylabel('Y')

Plot 2: Eigenvector visualization

axes[1].quiver(0, 0, eigenvectors[0, 0], eigenvectors[1, 0], angles='xy', scale_units='xy', scale=1, color='purple', width=0.015, label=f'Eigenvector 1 (λ={eigenvalues[0]:.2f})') axes[1].quiver(0, 0, eigenvectors[0, 1], eigenvectors[1, 1], angles='xy', scale_units='xy', scale=1, color='orange', width=0.015, label=f'Eigenvector 2 (λ={eigenvalues[1]:.2f})') axes[1].set_xlim(-1, 1) axes[1].set_ylim(-1, 1) axes[1].set_aspect('equal') axes[1].grid(True, alpha=0.3) axes[1].legend() axes[1].set_title('Eigenvectors (Principal Directions)') axes[1].set_xlabel('X') axes[1].set_ylabel('Y')

plt.tight_layout() plt.show()

print("\n✓ Linear algebra operations complete!") }, { id: 'calculus', category: 'Mathematics', title: '8. Calculus for ML', importance: 'Essential for Optimization', questions: ['What is a derivative?', 'What is gradient descent?', 'What is the chain rule?', 'What are partial derivatives?', 'How does backpropagation work?'], explanation:Calculus, especially derivatives, is crucial for training ML models. Gradient descent uses derivatives to optimize model parameters.

Key Concepts: • Derivatives: Rate of change • Gradient: Vector of partial derivatives • Chain Rule: Derivative of compositions (backpropagation) • Optimization: Finding minimum/maximum • Gradient Descent: Iterative optimization algorithm, code: import numpy as np import matplotlib.pyplot as plt

print("=== CALCULUS FOR ML ===\n")

1. Numerical Derivative

def f(x): """Function: f(x) = x^2 + 2x + 1""" return x**2 + 2*x + 1

def numerical_derivative(f, x, h=1e-5): """Compute derivative using limit definition""" return (f(x + h) - f(x)) / h

x_test = 3 derivative = numerical_derivative(f, x_test) analytical = 2*x_test + 2 # f'(x) = 2x + 2

print("1. DERIVATIVES") print(f"f(x) = x^2 + 2x + 1") print(f"f'({x_test}) numerical: {derivative:.6f}") print(f"f'({x_test}) analytical: {analytical:.6f}")

2. Gradient Descent (Core ML optimization!)

def cost_function(x): """Cost function to minimize: J(x) = x^2""" return x**2

def gradient(x): """Gradient of cost function: dJ/dx = 2x""" return 2*x

Gradient descent algorithm

x = 10.0 # Starting point learning_rate = 0.1 iterations = 20 history = [x]

print("\n2. GRADIENT DESCENT") print(f"Starting point: x = {x}")

for i in range(iterations): grad = gradient(x) x = x - learning_rate * grad # Update rule history.append(x) if i % 5 == 0: print(f"Iteration {i}: x = {x:.6f}, J(x) = {cost_function(x):.6f}")

print(f"Final: x = {x:.6f}, J(x) = {cost_function(x):.6f}")

3. Partial Derivatives (Multi-variable functions)

def f_multi(x, y): """f(x,y) = x^2 + 2xy + y^2""" return x**2 + 2xy + y^2""" return x**2 + 2xy + y2

def partial_x(x, y): """∂f/∂x = 2x + 2y""" return 2x + 2y

def partial_y(x, y): """∂f/∂y = 2x + 2y""" return 2x + 2y

x_val, y_val = 2, 3 print("\n3. PARTIAL DERIVATIVES") print(f"f(x,y) = x^2 + 2xy + y^2") print(f"At point ({x_val}, {y_val})😊 print(f"∂f/∂x = {partial_x(x_val, y_val)}") print(f"∂f/∂y = {partial_y(x_val, y_val)}")

4. Chain Rule (Used in backpropagation!)

def outer_function(u): """f(u) = u^2""" return u**2

def inner_function(x): """g(x) = 2x + 1""" return 2*x + 1

def composite(x): """h(x) = f(g(x)) = (2x + 1)^2""" return outer_function(inner_function(x))

x_val = 3

Chain rule: h'(x) = f'(g(x)) * g'(x)

u = inner_function(x_val) # u = g(x) df_du = 2*u # f'(u) = 2u dg_dx = 2 # g'(x) = 2 dh_dx = df_du * dg_dx # Chain rule

print("\n4. CHAIN RULE") print(f"h(x) = (2x + 1)²") print(f"h'({x_val}) = {dh_dx}") print("This is how backpropagation works in neural networks!")

5. Gradient Descent in 2D

def cost_2d(params): """J(x,y) = x^2 + y^2""" x, y = params return x**2 + y**2

def gradient_2d(params): """Gradient of J: [∂J/∂x, ∂J/∂y] = [2x, 2y]""" x, y = params return np.array([2*x, 2*y])

Run gradient descent

params = np.array([8.0, 6.0]) lr = 0.1 path = [params.copy()]

print("\n5. GRADIENT DESCENT (2D)") print(f"Starting point: {params}")

for i in range(30): grad = gradient_2d(params) params = params - lr * grad path.append(params.copy())

print(f"Final point: {params}") print(f"Final cost: {cost_2d(params):.6f}")

6. Learning Rate Effects

def run_gradient_descent(lr, iterations=20): x = 10.0 trajectory = [] for _ in range(iterations): trajectory.append(x) x = x - lr * gradient(x) return trajectory

lr_small = 0.01 lr_good = 0.1 lr_large = 0.5

print("\n6. LEARNING RATE IMPACT") print(f"Small LR (0.01): Final = {run_gradient_descent(lr_small)[-1]:.4f}") print(f"Good LR (0.1): Final = {run_gradient_descent(lr_good)[-1]:.4f}") print(f"Large LR (0.5): Final = {run_gradient_descent(lr_large)[-1]:.4f}")

Visualizations

fig = plt.figure(figsize=(16, 10))

Plot 1: Function and derivative

ax1 = plt.subplot(2, 3, 1) x_range = np.linspace(-5, 5, 100) y_range = f(x_range) y_deriv = 2*x_range + 2 ax1.plot(x_range, y_range, 'b-', linewidth=2, label='f(x) = x^2 + 2x + 1') ax1.plot(x_range, y_deriv, 'r--', linewidth=2, label="f'(x) = 2x + 2") ax1.axhline(y=0, color='k', linestyle='-', alpha=0.3) ax1.axvline(x=0, color='k', linestyle='-', alpha=0.3) ax1.legend() ax1.grid(True, alpha=0.3) ax1.set_title('Function and its Derivative') ax1.set_xlabel('x') ax1.set_ylabel('y')

Plot 2: Gradient descent trajectory (1D)

ax2 = plt.subplot(2, 3, 2) x_range = np.linspace(-2, 12, 100) y_range = cost_function(x_range) ax2.plot(x_range, y_range, 'b-', linewidth=2, label='J(x) = x^2') ax2.plot(history, [cost_function(x) for x in history], 'ro-', markersize=6, label='GD trajectory') ax2.legend() ax2.grid(True, alpha=0.3) ax2.set_title('Gradient Descent Optimization') ax2.set_xlabel('x') ax2.set_ylabel('J(x)')

Plot 3: Gradient descent in 2D

ax3 = plt.subplot(2, 3, 3) path_array = np.array(path) x_mesh = np.linspace(-10, 10, 100) y_mesh = np.linspace(-10, 10, 100) X, Y = np.meshgrid(x_mesh, y_mesh) Z = X**2 + Y**2 contour = ax3.contour(X, Y, Z, levels=20, alpha=0.6) ax3.clabel(contour, inline=True, fontsize=8) ax3.plot(path_array[:, 0], path_array[:, 1], 'ro-', markersize=8, linewidth=2, label='GD path') ax3.plot(path_array[0, 0], path_array[0, 1], 'go', markersize=12, label='Start') ax3.plot(path_array[-1, 0], path_array[-1, 1], 'r*', markersize=15, label='End') ax3.legend() ax3.set_title('Gradient Descent in 2D') ax3.set_xlabel('x') ax3.set_ylabel('y') ax3.grid(True, alpha=0.3)

Plot 4: Learning rate comparison

ax4 = plt.subplot(2, 3, 4) traj_small = run_gradient_descent(lr_small, 50) traj_good = run_gradient_descent(lr_good, 20) traj_large = run_gradient_descent(lr_large, 20) ax4.plot(traj_small, 'b-', label='LR=0.01 (too small)') ax4.plot(traj_good, 'g-', label='LR=0.1 (good)') ax4.plot(traj_large, 'r-', label='LR=0.5 (too large)') ax4.axhline(y=0, color='k', linestyle='--', alpha=0.3, label='Minimum') ax4.legend() ax4.grid(True, alpha=0.3) ax4.set_title('Impact of Learning Rate') ax4.set_xlabel('Iteration') ax4.set_ylabel('x value')

Plot 5: Tangent line visualization

ax5 = plt.subplot(2, 3, 5) x_point = 2 y_point = f(x_point) slope = 2*x_point + 2 x_range = np.linspace(-1, 5, 100) tangent = slope * (x_range - x_point) + y_point ax5.plot(x_range, f(x_range), 'b-', linewidth=2, label='f(x)') ax5.plot(x_range, tangent, 'r--', linewidth=2, label='Tangent line') ax5.plot(x_point, y_point, 'go', markersize=10, label=f'Point ({x_point}, {y_point:.1f})') ax5.legend() ax5.grid(True, alpha=0.3) ax5.set_title(f'Tangent Line (slope = {slope})') ax5.set_xlabel('x') ax5.set_ylabel('y')

Plot 6: Convex vs Non-convex

ax6 = plt.subplot(2, 3, 6) x_range = np.linspace(-3, 3, 100) convex = x_range**2 non_convex = np.sin(2*x_range) + 0.1*x_range**2 ax6.plot(x_range, convex, 'b-', linewidth=2, label='Convex (easy to

optimize)') ax6.plot(x_range, non_convex, 'r-', linewidth=2, label='Non-convex (harder)') ax6.legend() ax6.grid(True, alpha=0.3) ax6.set_title('Convex vs Non-convex Functions') ax6.set_xlabel('x') ax6.set_ylabel('y')

plt.tight_layout() plt.show()

print("\n✓ Calculus concepts visualized!") }, { id: 'data-preprocessing', category: 'Data Processing', title: '9. Data Preprocessing', importance: 'Critical for Model Performance', questions: ['What is feature scaling and why is it important?', 'How to handle missing values?', 'What is one-hot encoding?', 'What is train-test split?', 'How to detect and handle outliers?'], explanation:Data preprocessing is 80% of ML work. Clean, well-prepared data is more important than complex models.

Key Concepts: • Missing Data: Imputation strategies • Feature Scaling: Normalization, standardization • Encoding: Convert categorical to numerical • Train-Test Split: Prevent overfitting • Outlier Detection: Identify anomalies, code: import numpy as np import pandas as pd from sklearn.preprocessing import StandardScaler, MinMaxScaler, LabelEncoder from sklearn.model_selection import train_test_split from sklearn.impute import SimpleImputer import matplotlib.pyplot as plt import seaborn as sns

print("=== DATA PREPROCESSING ===\n")

Create sample dataset with common issues

np.random.seed(42) data = pd.DataFrame({ 'Age': [25, 30, np.nan, 35, 40, 22, np.nan, 28, 45, 33], 'Salary': [50000, 60000, 55000, 80000, 90000, 45000, 52000, np.nan, 95000, 70000], 'Experience': [2, 5, 3, 8, 12, 1, 3, 4, 15, 7], 'City': ['NYC', 'LA', 'NYC', 'Chicago', 'LA', 'NYC', 'Chicago', 'LA', 'NYC', 'Chicago'], 'Purchased': ['Yes', 'No', 'Yes', 'Yes', 'No', 'No', 'Yes', 'Yes', 'No', 'Yes'] })

print("1. ORIGINAL DATA WITH ISSUES") print(data) print(f"\nMissing values:\n{data.isnull().sum()}")

2. Handling Missing Data

print("\n2. HANDLING MISSING DATA")

Strategy 1: Mean imputation for numerical

imputer_mean = SimpleImputer(strategy='mean') data['Age'] = imputer_mean.fit_transform(data[['Age']]) print("After mean imputation for Age:") print(data['Age'].values)

Strategy 2: Median imputation (robust to outliers)

imputer_median = SimpleImputer(strategy='median') data['Salary'] = imputer_median.fit_transform(data[['Salary']]) print("\nAfter median imputation for Salary:") print(data['Salary'].values)

3. Feature Scaling

print("\n3. FEATURE SCALING") print("Original Salary and Age (different scales)😊 print(data[['Age', 'Salary']].head())

Standardization (z-score): mean=0, std=1

scaler_standard = StandardScaler() data[['Age_scaled', 'Salary_scaled']] = scaler_standard.fit_transform(data[['Age', 'Salary']]) print("\nAfter Standardization:") print(data[['Age_scaled', 'Salary_scaled']].head()) print(f"Mean: {data['Age_scaled'].mean():.6f}, Std: {data['Age_scaled'].std():.6f}")

Min-Max Normalization: scale to [0, 1]

scaler_minmax = MinMaxScaler() data[['Age_normalized', 'Salary_normalized']] = scaler_minmax.fit_transform(data[['Age', 'Salary']]) print("\nAfter Min-Max Normalization:") print(data[['Age_normalized', 'Salary_normalized']].head())

4. Encoding Categorical Variables

print("\n4. CATEGORICAL ENCODING")

Label Encoding (for ordinal or binary)

le = LabelEncoder() data['Purchased_encoded'] = le.fit_transform(data['Purchased']) print("Label Encoding (Purchased)😊 print(pd.DataFrame({'Original': data['Purchased'], 'Encoded': data['Purchased_encoded']}))

One-Hot Encoding (for nominal categories)

data_encoded = pd.get_dummies(data, columns=['City'], prefix='City') print("\nOne-Hot Encoding (City) 😊 print(data_encoded[['City_Chicago', 'City_LA', 'City_NYC']].head())

5. Outlier Detection

print("\n5. OUTLIER DETECTION")

Method 1: IQR method

Q1 = data['Salary'].quantile(0.25) Q3 = data['Salary'].quantile(0.75) IQR = Q3 - Q1 lower_bound = Q1 - 1.5 * IQR upper_bound = Q3 + 1.5 * IQR

outliers_iqr = data[(data['Salary'] < lower_bound) | (data['Salary'] > upper_bound)] print(f"IQR Method: Lower={lower_bound:.2f}, Upper={upper_bound:.2f}") print(f"Number of outliers: {len(outliers_iqr)}")

Method 2: Z-score method

z_scores = np.abs((data['Salary'] - data['Salary'].mean()) / data['Salary'].std()) outliers_zscore = data[z_scores > 3] print(f"\nZ-score Method (|z| > 3)😊 print(f"Number of outliers: {len(outliers_zscore)}")

6. Train-Test Split

print("\n6. TRAIN-TEST SPLIT") X = data_encoded[['Age', 'Salary', 'Experience', 'City_Chicago', 'City_LA', 'City_NYC']] y = data['Purchased_encoded']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

print(f"Training set: {len(X_train)} samples") print(f"Test set: {len(X_test)} samples") print(f"Target distribution in train: {np.bincount(y_train)}") print(f"Target distribution in test: {np.bincount(y_test)}")

7. Feature Engineering

print("\n7. FEATURE ENGINEERING") data['Salary_per_Experience'] = data['Salary'] / (data['Experience'] + 1) data['Age_Group'] = pd.cut(data['Age'], bins=[0, 30, 40, 100], labels=['Young', 'Middle', 'Senior']) print("New engineered features:") print(data[['Salary_per_Experience', 'Age_Group']].head())

Visualizations

fig, axes = plt.subplots(2, 3, figsize=(16, 10))

Plot 1: Distribution before and after scaling

axes[0, 0].hist(data['Age'], bins=10, alpha=0.5, label='Original Age', color='blue') axes[0, 0].hist(data['Age_scaled']*10 + 30, bins=10, alpha=0.5, label='Scaled Age (rescaled for viz)', color='red') axes[0, 0].set_title('Scaling Comparison') axes[0, 0].legend() axes[0, 0].set_xlabel('Value') axes[0, 0].set_ylabel('Frequency')

Plot 2: Correlation heatmap

corr_data = data[['Age', 'Salary', 'Experience']].corr() sns.heatmap(corr_data, annot=True, cmap='coolwarm', center=0, square=True, ax=axes[0, 1]) axes[0, 1].set_title('Correlation Matrix')

Plot 3: Box plot for outliers

axes[0, 2].boxplot(data['Salary'], vert=True) axes[0, 2].axhline(y=lower_bound, color='r', linestyle='--', label='Lower bound') axes[0, 2].axhline(y=upper_bound, color='r', linestyle='--', label='Upper bound') axes[0, 2].set_title('Outlier Detection (Box Plot)') axes[0, 2].set_ylabel('Salary') axes[0, 2].legend()

Plot 4: One-hot encoding visualization

city_counts = data['City'].value_counts() axes[1, 0].bar(city_counts.index, city_counts.values, color=['#FF6B6B', '#4ECDC4', '#45B7D1']) axes[1, 0].set_title('Categorical Distribution') axes[1, 0].set_xlabel('City') axes[1, 0].set_ylabel('Count')

Plot 5: Before vs After standardization

axes[1, 1].scatter(data['Age'], data['Salary'], s=100, alpha=0.6, c='blue', label='Original') axes[1, 1].set_title('Original Scale') axes[1, 1].set_xlabel('Age') axes[1, 1].set_ylabel('Salary') axes[1, 1].legend()

axes[1, 2].scatter(data['Age_scaled'], data['Salary_scaled'], s=100, alpha=0.6, c='red', label='Standardized') axes[1, 2].set_title('After Standardization') axes[1, 2].set_xlabel('Age (scaled)') axes[1, 2].set_ylabel('Salary (scaled)') axes[1, 2].legend()

plt.tight_layout() plt.show()

print("\n✓ Data preprocessing complete!") , }, { id: 'ml-basics', category: 'ML Fundamentals', title: '10. Machine Learning Basics', importance: 'Core ML Concepts', questions: ['What is supervised vs unsupervised learning?', 'What is overfitting and underfitting?', 'What is bias-variance tradeoff?', 'What is cross-validation?', 'How to evaluate model performance?'], explanation: Understanding ML fundamentals is crucial before diving into algorithms.

Key Concepts: • Supervised: Learn from labeled data (Classification, Regression) • Unsupervised: Find patterns in unlabeled data (Clustering, Dimensionality Reduction) • Overfitting: Model memorizes training data • Underfitting: Model too simple to capture patterns • Validation: Test model on unseen data, code: import numpy as np import matplotlib.pyplot as plt from sklearn.model_selection import cross_val_score, learning_curve from sklearn.linear_model import LinearRegression, Ridge from sklearn.preprocessing import PolynomialFeatures from sklearn.metrics import mean_squared_error, r2_score from sklearn.tree import DecisionTreeRegressor

print("=== MACHINE LEARNING BASICS ===\n")

Generate synthetic dataset

np.random.seed(42) X = np.linspace(0, 10, 50).reshape(-1, 1) y = 2 * X.ravel() + 3 * np.sin(X.ravel()) + np.random.randn(50) * 0.5

Split data

split_idx = 40 X_train, X_test = X[:split_idx], X[split_idx:] y_train, y_test = y[:split_idx], y[split_idx:]

print("1. SUPERVISED LEARNING - REGRESSION") print(f"Training samples: {len(X_train)}") print(f"Test samples: {len(X_test)}")

Model 1: Underfitting (too simple)

model_underfit = LinearRegression() model_underfit.fit(X_train, y_train) y_pred_under = model_underfit.predict(X_test) rmse_under = np.sqrt(mean_squared_error(y_test, y_pred_under))

print(f"\nUnderfitting Model (Linear)😊 print(f" Train R^2: {model_underfit.score(X_train, y_train):.4f}") print(f" Test R^2: {model_underfit.score(X_test, y_test):.4f}") print(f" Test RMSE: {rmse_under:.4f}")

Model 2: Good fit (balanced complexity)

poly_good = PolynomialFeatures(degree=3) X_train_poly = poly_good.fit_transform(X_train) X_test_poly = poly_good.transform(X_test)

model_goodfit = Ridge(alpha=1.0) model_goodfit.fit(X_train_poly, y_train) y_pred_good = model_goodfit.predict(X_test_poly) rmse_good = np.sqrt(mean_squared_error(y_test, y_pred_good))

print(f"\nGood Fit Model (Polynomial deg=3 + Ridge)😊 print(f" Train R^2: {model_goodfit.score(X_train_poly, y_train):.4f}") print(f" Test R^2: {model_goodfit.score(X_test_poly, y_test):.4f}") print(f" Test RMSE: {rmse_good:.4f}")

Model 3: Overfitting (too complex)

model_overfit = DecisionTreeRegressor(max_depth=15, random_state=42) model_overfit.fit(X_train, y_train) y_pred_over = model_overfit.predict(X_test) rmse_over = np.sqrt(mean_squared_error(y_test, y_pred_over))

print(f"\nOverfitting Model (Deep Decision Tree)😊 print(f" Train R^2: {model_overfit.score(X_train, y_train):.4f}") print(f" Test R^2: {model_overfit.score(X_test, y_test):.4f}") print(f" Test RMSE: {rmse_over:.4f}")

2. Bias-Variance Tradeoff

print("\n2. BIAS-VARIANCE TRADEOFF") print("High Bias (Underfitting): Model too simple, poor train & test performance") print("High Variance (Overfitting): Model too complex, great train but poor test") print("Balanced: Good performance on both train and test sets")

3. Cross-Validation

print("\n3. CROSS-VALIDATION (K-Fold)") from sklearn.model_selection import KFold

kfold = KFold(n_splits=5, shuffle=True, random_state=42) cv_scores = cross_val_score(model_goodfit, X_train_poly, y_train, cv=kfold, scoring='r2')

print(f"5-Fold CV Scores: {cv_scores}") print(f"Mean CV Score: {cv_scores.mean():.4f} (+/- {cv_scores.std():.4f})") print("CV helps estimate how well model generalizes to unseen data")

4. Learning Curves

print("\n4. LEARNING CURVES") train_sizes, train_scores, val_scores = learning_curve(Ridge(alpha=1.0), X_train_poly, y_train, cv=5, train_sizes=np.linspace(0.1, 1.0, 10), scoring='r2', random_state=42)

train_mean = train_scores.mean(axis=1) val_mean = val_scores.mean(axis=1)

print(f"With more training data:") print(f" Train score improves: {train_mean[0]:.3f} -> {train_mean[-1]:.3f}") print(f" Val score improves: {val_mean[0]:.3f} -> {val_mean[-1]:.3f}")

Visualizations

fig, axes = plt.subplots(2, 2, figsize=(15, 12))

Plot 1: Underfitting, Good Fit, Overfitting

X_plot = np.linspace(0, 10, 200).reshape(-1, 1)

axes[0, 0].scatter(X_train, y_train, c='blue', s=50, alpha=0.6, label='Train') axes[0, 0].scatter(X_test, y_test, c='red', s=50, alpha=0.6, label='Test') axes[0, 0].plot(X_plot, model_underfit.predict(X_plot), 'g-', linewidth=2, label=f'Underfit (RMSE={rmse_under:.2f})') axes[0, 0].set_title('Underfitting: High Bias') axes[0, 0].legend() axes[0, 0].grid(True, alpha=0.3)

axes[0, 1].scatter(X_train, y_train, c='blue', s=50, alpha=0.6, label='Train') axes[0, 1].scatter(X_test, y_test, c='red', s=50, alpha=0.6, label='Test') X_plot_poly = poly_good.transform(X_plot) axes[0, 1].plot(X_plot, model_goodfit.predict(X_plot_poly), 'g-', linewidth=2, label=f'Good Fit (RMSE={rmse_good:.2f})') axes[0, 1].set_title('Good Fit: Balanced') axes[0, 1].legend() axes[0, 1].grid(True, alpha=0.3)

axes[1, 0].scatter(X_train, y_train, c='blue', s=50, alpha=0.6, label='Train') axes[1, 0].scatter(X_test, y_test, c='red', s=50, alpha=0.6, label='Test') axes[1, 0].plot(X_plot, model_overfit.predict(X_plot), 'g-', linewidth=2, label=f'Overfit (RMSE={rmse_over:.2f})') axes[1, 0].set_title('Overfitting: High Variance') axes[1, 0].legend() axes[1, 0].grid(True, alpha=0.3)

Plot 2: Learning curves

axes[1, 1].plot(train_sizes, train_mean, 'o-', color='blue', linewidth=2, label='Training score') axes[1, 1].plot(train_sizes, val_mean, 'o-', color='red', linewidth=2, label='Validation score') axes[1, 1].fill_between(train_sizes, train_scores.mean(axis=1) - train_scores.std(axis=1), train_scores.mean(axis=1) + train_scores.std(axis=1), alpha=0.1, color='blue') axes[1, 1].fill_between(train_sizes, val_scores.mean(axis=1) - val_scores.std(axis=1), val_scores.mean(axis=1) + val_scores.std(axis=1), alpha=0.1, color='red') axes[1, 1].set_xlabel('Training Set Size') axes[1, 1].set_ylabel('R^2 Score') axes[1, 1].set_title('Learning Curves') axes[1, 1].legend(loc='best') axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout() plt.show()

Summary table

print("\n5. MODEL COMPARISON SUMMARY") print("="*60) print(f"{'Model':<20} {'Train R^2':<12} {'Test R^2':<12} {'Status':<15}") print("="*60) print(f"{'Linear (Underfit)':<20} {model_underfit.score(X_train, y_train):^12.4f} " f" {model_underfit.score(X_test, y_test):^12.4f} {'High Bias':<15}") print(f"{'Polynomial+Ridge':<20} {model_goodfit.score(X_train_poly, y_train):^12.4f} " f"{model_goodfit.score(X_test_poly, y_test):^12.4f} {'Balanced':<15}") print(f"{'Deep Tree (Overfit)':<20} {model_overfit.score(X_train, y_train):^12.4f} " f" {model_overfit.score(X_test, y_test):^12.4f} {'High Variance':<15}") print("="*60)

print("\n✓ ML basics demonstration complete!") print("\nKEY TAKEAWAYS:") print("• Underfitting: Both train and test scores are poor") print("• Overfitting: Train score great, test score poor") print("• Good fit: Both scores are good and similar") print("• Use cross-validation to reliably estimate performance") print("• Learning curves show if more data will help")`